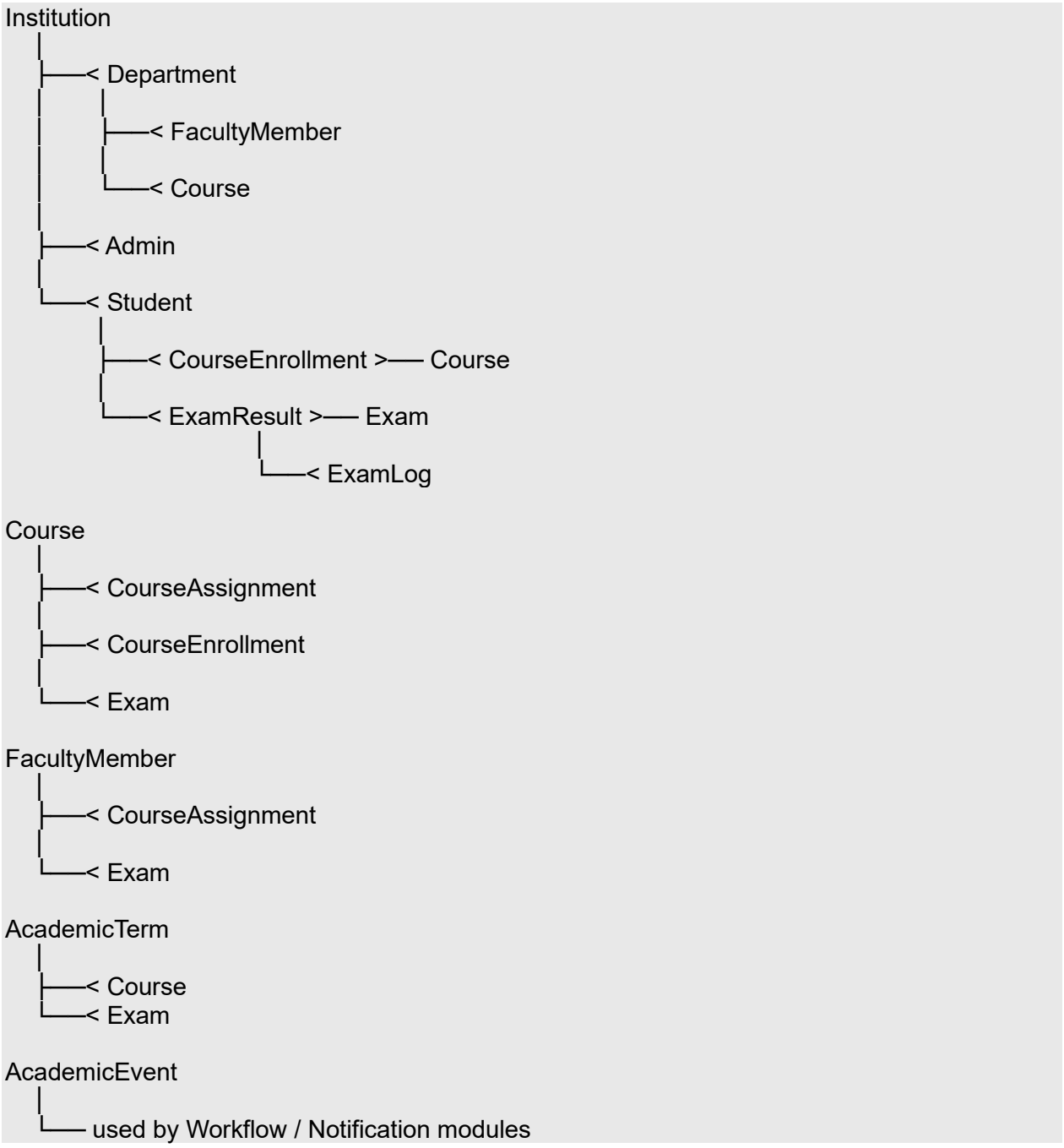
Recommended Academic ERD

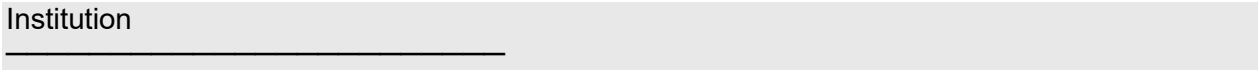
I would structure it like this:



But I would make a few important adjustments to your original idea.

1. Institution

Represents the educational institution using SACP.



```
Id : Guid PK
Name : string
Code : string
Description : string?
Email : string?
Phone : string?
Address : string?
Website : string?
IsActive : bool
CreatedAt : DateTime
UpdatedAt : DateTime?
```

Relationships

```
Institution 1 ——— * Department
Institution 1 ——— * Student
Institution 1 ——— * FacultyMember
Institution 1 ——— * Admin
Institution 1 ——— * AcademicTerm
```

2. Department

Represents academic departments.

```
Department
————————————————————————
Id : Guid PK
InstitutionId : Guid FK
Name : string
Code : string
Description : string?
HeadFacultyMemberId : Guid? FK
IsActive : bool
CreatedAt : DateTime
UpdatedAt : DateTime?
```

Relationships

```
Institution 1 ——— * Department

Department 1 ——— * Student
Department 1 ——— * FacultyMember
Department 1 ——— * Course

Department 0..1 ——— 1 FacultyMember
                  ↑
            Head of Department
```

I would **not** create a separate DepartmentHead entity.

A department head is simply a faculty member assigned to that responsibility.

3. Admin

Here's an important architectural point.

If your **User entity already exists in Identity**, don't duplicate authentication information inside Admin.

Instead:

```
Admin
──────────────────────────────
Id : Guid PK/FK → User
InstitutionId : Guid FK
DepartmentId : Guid? FK
AdminLevel : enum
CreatedAt : DateTime
```

For example:

```
AdminLevel
──────────────────
Institution
Department
System
```

However, if your existing SACP architecture uses **User + Role**, I'd actually recommend:

```
User
  │
  └── Role = Admin
```

and only create an AdminProfile if admins need academic-specific information.

So your ERD can simply show:

```
User
  │
  └── Admin Profile
```

rather than duplicating the entire user table.

4. FacultyMember

```
FacultyMember
──────────────────────────────
Id : Guid PK/FK → User
InstitutionId : Guid FK
DepartmentId : Guid FK
EmployeeCode : string
Designation : string?
JoiningDate : DateOnly?
IsActive : bool
```

Relationships

```
Institution 1 ──── * FacultyMember
Department 1 ──── * FacultyMember

FacultyMember 1 ──── * CourseAssignment
FacultyMember 1 ──── * Exam
```

5. Student

```
Student
──────────────────────────────
Id : Guid PK/FK → User
InstitutionId : Guid FK
DepartmentId : Guid FK
StudentCode : string
EnrollmentDate : DateOnly
CurrentSemester : int?
AcademicStatus : enum
IsActive : bool
```

Possible:

```
AcademicStatus
──────────────────────
Active
Graduated
Suspended
Withdrawn
Inactive
```

Relationships

```
Institution 1 ──── * Student
Department 1 ──── * Student

Student 1 ──── * CourseEnrollment
Student 1 ──── * ExamResult
```

6. AcademicTerm

I strongly recommend adding this.

Without it, you'll eventually have trouble distinguishing:

```
Database Systems — Fall 2026
Database Systems — Spring 2027
Database Systems — Fall 2027
```

You don't need a full semester-management system. Just enough context.

```
AcademicTerm
──────────────────────────────
Id : Guid PK
InstitutionId : Guid FK
Name : string
Code : string
StartDate : DateOnly
EndDate : DateOnly
IsCurrent : bool
```

Examples:

- Fall 2026
- Spring 2027
- Summer 2027

Relationships

```
Institution 1 ——— * AcademicTerm

AcademicTerm 1 ——— * Course
AcademicTerm 1 ——— * CourseAssignment
AcademicTerm 1 ——— * Exam
```

7. Course

Don't make Course represent a particular offering.

Keep the course itself relatively stable.

```
Course
————————————————————————
Id : Guid PK
InstitutionId : Guid FK
DepartmentId : Guid FK
Code : string
Name : string
Description : string?
CreditHours : decimal?
IsActive : bool
```

Example:

- CS-301
- Database Systems

Relationships

```
Department 1 ——— * Course

Course 1 ——— * CourseAssignment
Course 1 ——— * CourseEnrollment
Course 1 ——— * Exam
```

8. CourseAssignment

This is one of your most important entities.

It represents:

Which faculty member teaches which course during which academic term?

```
CourseAssignment
————————————————————————
Id : Guid PK
CourseId : Guid FK
FacultyMemberId : Guid FK
AcademicTermId : Guid FK
Section : string?
AssignedAt : DateTime
IsActive : bool
```

Relationship:

```
Course 1 ———————— *
FacultyMember 1 ——— * CourseAssignment
AcademicTerm 1 ——— *
```

This avoids putting:

FacultyMemberId

directly inside Course.

9. CourseEnrollment

Represents student enrollment in a course offering.

```
CourseEnrollment
————————————————————————
Id : Guid PK
CourseId : Guid FK
StudentId : Guid FK
AcademicTermId : Guid FK
EnrollmentDate : DateOnly
Status : enum
FinalGrade : string?
```

Possible:

```
EnrollmentStatus
————————————————
Enrolled
Dropped
Completed
Withdrawn
```

Relationship

```
Student 1 ————————— *
Course 1 ———————————— *
AcademicTerm 1 ——— *
```

This gives you:

```
Student
↓
CourseEnrollment
↓
Course
↓
FacultyMember
```

which is exactly the academic context your communication engine needs.

10. Exam

Keep this simple.

You don't need a complete examination management system.

```
Exam
```

```
————————————————————
Id : Guid PK
CourseId : Guid FK
AcademicTermId : Guid FK
Title : string
ExamType : enum
ExamDate : DateTime
DurationMinutes : int?
Location : string?
MaxMarks : decimal?
Status : enum
CreatedAt : DateTime
```

Possible:

```
ExamType
————————————
Quiz
Midterm
Final
Practical
Assignment
Other
```

Relationships

```
Course 1 ——————— * Exam
AcademicTerm 1 —— * Exam

Exam 1 ———————————— * ExamResult
Exam 1 ———————————— * ExamLog
```

11. ExamResult

I would **definitely add this**, rather than having only ExamLogs.

Because these represent two completely different concepts.

```
ExamResult
————————————————————
Id : Guid PK
ExamId : Guid FK
StudentId : Guid FK
MarksObtained : decimal
Grade : string?
Status : enum
PublishedAt : DateTime?
```

Relationships:

```
Exam 1 ——————— * ExamResult
Student 1 ——— * ExamResult
```

This gives SACP useful events such as:

```
Exam result published
↓
```

```
Student notification
```

or potentially:

```
Low result
↓
Academic intervention workflow
```

12. ExamLog

Keep your proposed ExamLog, but use it for **audit/history**, not results.

```
ExamLog
————————————————————
Id : Guid PK
ExamId : Guid FK
Action : enum
PerformedBy : Guid FK → User
Timestamp : DateTime
Description : string?
```

Example actions:

```
Created
Updated
Rescheduled
Cancelled
Published
ResultPublished
```

Relationship:

```
Exam 1 ———————— * ExamLog
User 1 ———————— * ExamLog
```

This is valuable for your communication system because:

```
Exam rescheduled
↓
Academic Event
↓
Workflow
↓
Notification
```

13. AcademicEvent

This is the **one entity I'd particularly recommend for SACP**.

It should not become another huge ERP module.

It's essentially a normalized representation of something academically relevant that can trigger communication.

```
AcademicEvent
————————————————————
Id : Guid PK
InstitutionId : Guid FK
AcademicTermId : Guid? FK
```

```
CourseId : Guid? FK
EventType : enum
Title : string
Description : string?
StartAt : DateTime
EndAt : DateTime?
SourceEntityId : Guid?
CreatedAt : DateTime
```

Possible:

```
EventType
_____________________

ExamScheduled
ExamRescheduled
ExamCancelled
AssignmentDeadline
AttendanceThreshold
ResultPublished
CourseAnnouncement
AcademicDeadline
```

Why this is useful

Your workflow engine can eventually consume:

```
AcademicEvent
      ↓
Workflow Rule
      ↓
Notification
```

instead of your workflow engine having to understand every academic entity individually.

That is **very useful architecture for SACP**.

14. AcademicCalendarEvent

I'd only add this if you want actual institution-wide events such as:

- Orientation Day
- Holiday
- Admission Deadline
- Semester Start
- Semester End

Then:

```
AcademicCalendarEvent
_____________________________________

Id : Guid PK
InstitutionId : Guid FK
AcademicTermId : Guid? FK
Title : string
Description : string?
```

```
StartDate : DateTime
EndDate : DateTime?
EventType : enum
IsPublic : bool
```

But **you don't necessarily need this in V1**.

If you already have AcademicEvent, these can simply be represented as:

```
AcademicEvent.EventType = InstitutionalEvent
```

I'd initially **avoid the extra entity**.

15. Attendance

Here's another important consideration.

You specifically mentioned:

"Alerts about short attendance"

You can't properly support that with only Student + Course.

You need some representation of attendance.

But don't build a complete attendance-management system.

I'd use:

```
AttendanceRecord
_____________________________
Id : Guid PK
StudentId : Guid FK
CourseId : Guid FK
AcademicTermId : Guid FK
Date : DateOnly
Status : enum
RecordedBy : Guid FK
RecordedAt : DateTime
```

Possible:

- Present
- Absent
- Late
- Excused

Relationship:

```
Student 1 ———————— *
Course 1 ———————— *
AcademicTerm 1 —— *
FacultyMember 1 — *
```

Then SACP can calculate:

```
Attendance %
```

```
        ↓
Below threshold?
        ↓
  AcademicEvent
        ↓
    Workflow
        ↓
  Notification
```

This is far more useful for your project than adding things like Fee, Payroll, Library, Hostel, etc.

16. Assignment / AcademicTask

Your proposal has assignment/deadline notifications.

You could technically generate these from generic AcademicEvent, but I'd add a lightweight entity:

```
AcademicTask
————————————————————————
Id : Guid PK
CourseId : Guid FK
FacultyMemberId : Guid FK
AcademicTermId : Guid FK
Title : string
Description : string?
DueDate : DateTime
CreatedAt : DateTime
Status : enum
```

Notice what we're **not** implementing:

- Submission
- FileUpload
- Grading
- Rubrics
- Plagiarism

That's LMS territory.

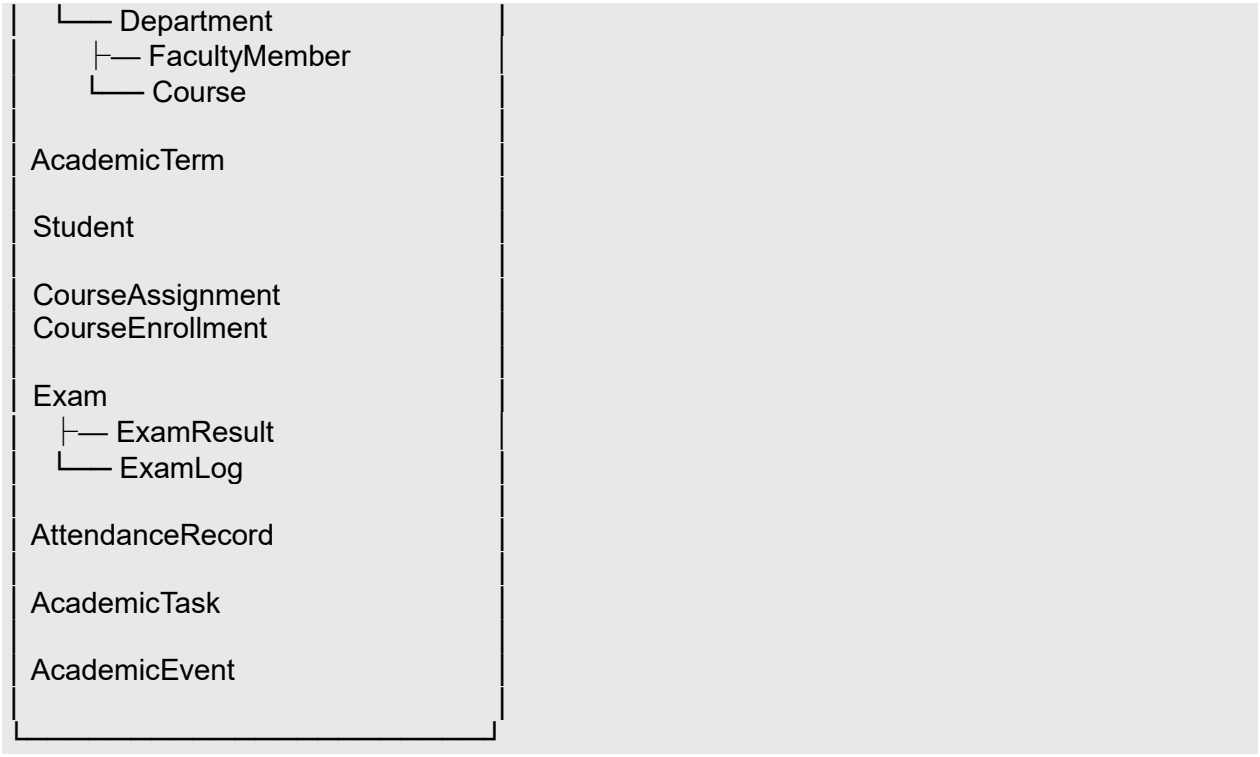
We're only representing the academic task sufficiently to generate:

"Assignment XYZ is due tomorrow."

Final Entity Set

So I'd recommend **16 entities**:

```
┌──────────────────────────────┐
│       ACADEMIC MODULE        │
├──────────────────────────────┤
│                              │
│  Institution                 │
```



Plus your existing **User** entity from Identity.

Complete Relationship Matrix

This is probably the most useful thing to use while creating your ERD.

Entity A	***Relationship***	***Entity B***
Institution	1 → M	Department
Institution	1 → M	Student
Institution	1 → M	FacultyMember
Institution	1 → M	Admin/Profile
Institution	1 → M	AcademicTerm
Institution	1 → M	Course
Department	1 → M	Student
Department	1 → M	FacultyMember
Department	1 → M	Course
Department	0..1 → 1	FacultyMember (HOD)
User	1 → 0..1	Student

User	1 → 0..1	FacultyMember
User	1 → 0..1	Admin
AcademicTerm	1 → M	CourseAssignment
AcademicTerm	1 → M	CourseEnrollment
AcademicTerm	1 → M	Exam
Course	1 → M	CourseAssignment
FacultyMember	1 → M	CourseAssignment
Student	1 → M	CourseEnrollment
Course	1 → M	CourseEnrollment
Course	1 → M	Exam
Exam	1 → M	ExamResult
Student	1 → M	ExamResult
Exam	1 → M	ExamLog
User	1 → M	ExamLog
Student	1 → M	AttendanceRecord
Course	1 → M	AttendanceRecord
AcademicTerm	1 → M	AttendanceRecord
FacultyMember	1 → M	AttendanceRecord
Course	1 → M	AcademicTask
FacultyMember	1 → M	AcademicTask
AcademicTerm	1 → M	AcademicTask
Institution	1 → M	AcademicEvent
AcademicTerm	0..1 → M	AcademicEvent
Course	0..1 → M	AcademicEvent

One Architectural Decision I'd Make Now

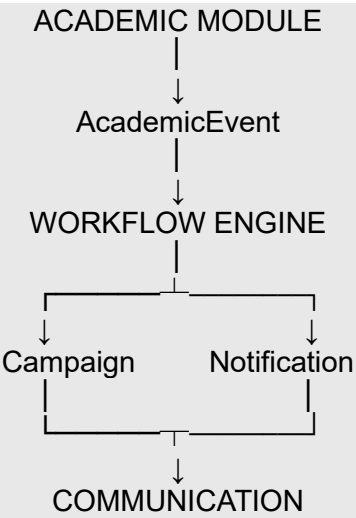
Keep **communication entities OUT of this Academic ERD**.

Don't connect:

Student → Notification
Course → Campaign
Exam → Email

directly everywhere.

Instead:



This separation is **very important for SACP**.

The Academic Module provides **academic context**.

The Workflow Engine decides **what should happen**.

The Communication Module decides **how to communicate it**.

And the AI Layer can decide **how important the communication is / how it should be processed**.

That gives you a reasonably sized Academic ERD without turning SACP into a full ERP or LMS.